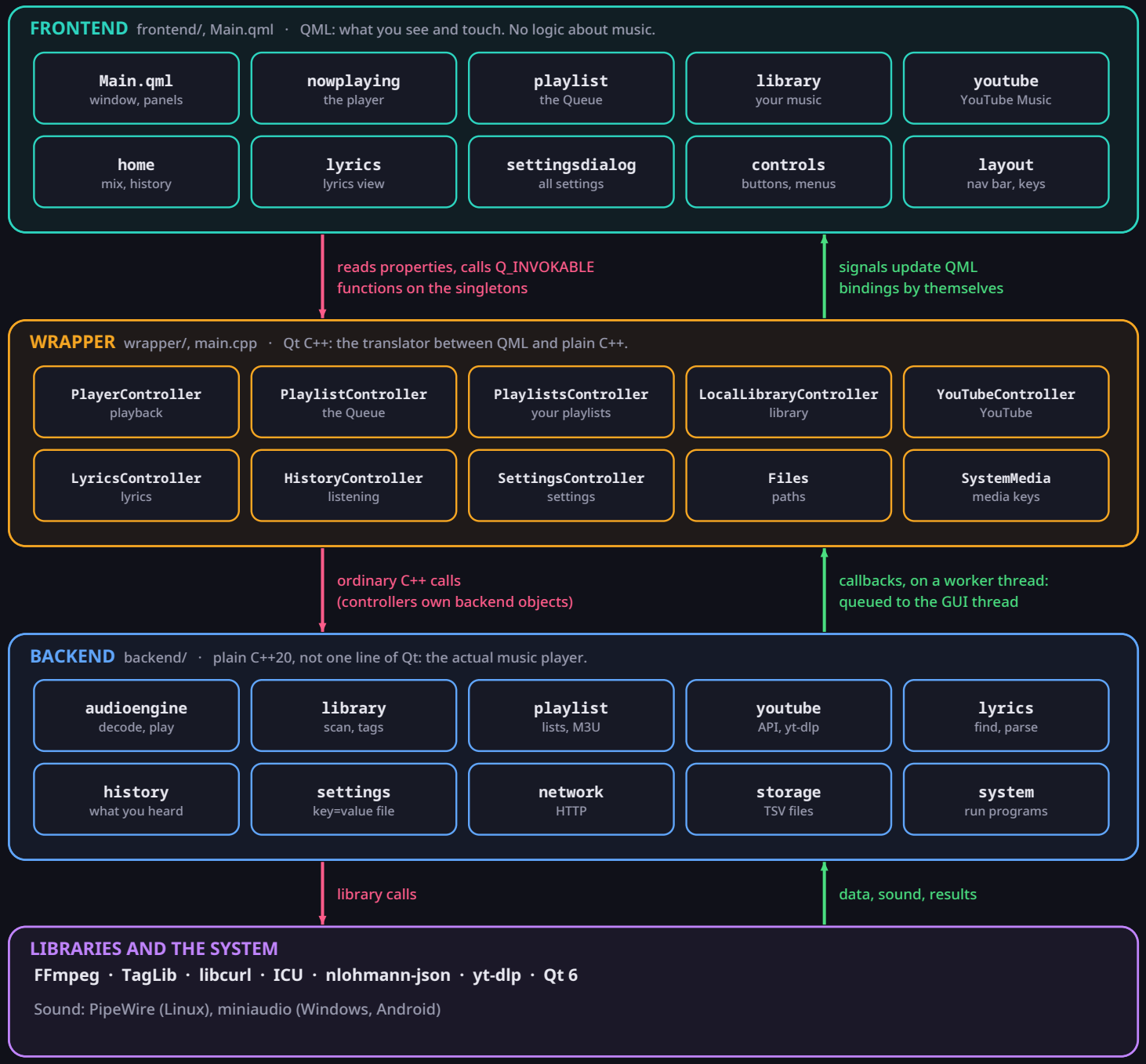
The big picture

Three layers, each talking only to the one next to it. A change in one layer stays in that layer.



The rules that keep it clean

The backend knows nothing about Qt.
No backend file includes a Qt header, so it could be tested or reused alone.

QML never reaches the backend.
Every screen goes through a wrapper singleton, such as PlayerController.

Results always return to the GUI thread.
Player and Scanner have their own threads, and controllers use a thread pool. QML may only be touched on the GUI thread, so the wrapper moves every result there.

Platform code is fenced off.
Only a few files differ per platform, behind one shared interface (page 6).

Slow work never blocks the screen.
Network, scanning and yt-dlp run on worker threads.

What is what

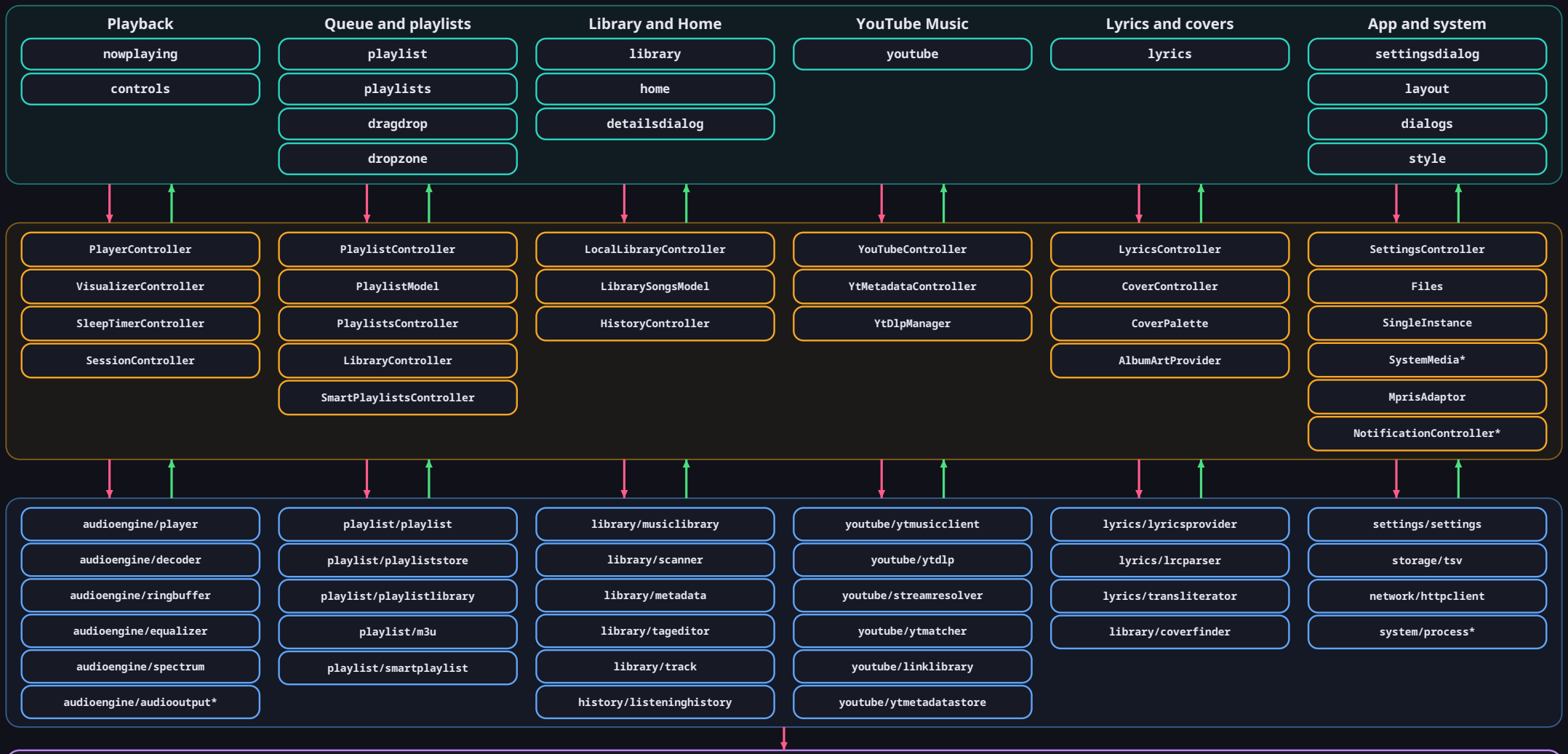
QML	a language that describes screens
Controller	a C++ class QML can talk to
Singleton	one shared instance, used by name
Signal	"something changed", sent by a class
Binding	QML value that updates by itself
Q_INVOKABLE	a C++ function QML may call

In numbers

QML files	**63**
wrapper C++ files	**55**
backend C++ files	**62**

The blueprint

Read each column top to bottom: the screens, the wrapper classes they use, the backend modules behind them.



LIBRARIES

FFmpeg decoder · **PipeWire / miniaudio** audiooutput · **TagLib** metadata, tageditor, lyricsprovider · **libcurl** httpclient · **ICU** transliterator · **nlohmann-json** ytmusicclient, ytdlp, coverfinder, lyricsprovider, smartplaylist · **yt-dlp** ytdlp, through system/process

Reading it

→ calls into the layer below

← reports back up

* one .cpp per platform behind one interface (page 6)

The columns group by subject. Controllers also use each other, and shared modules reach across columns: page 3 lists them class by class.

Tying it together: main.cpp creates all the wrapper classes and loads Main.qml (page 5).

The connections

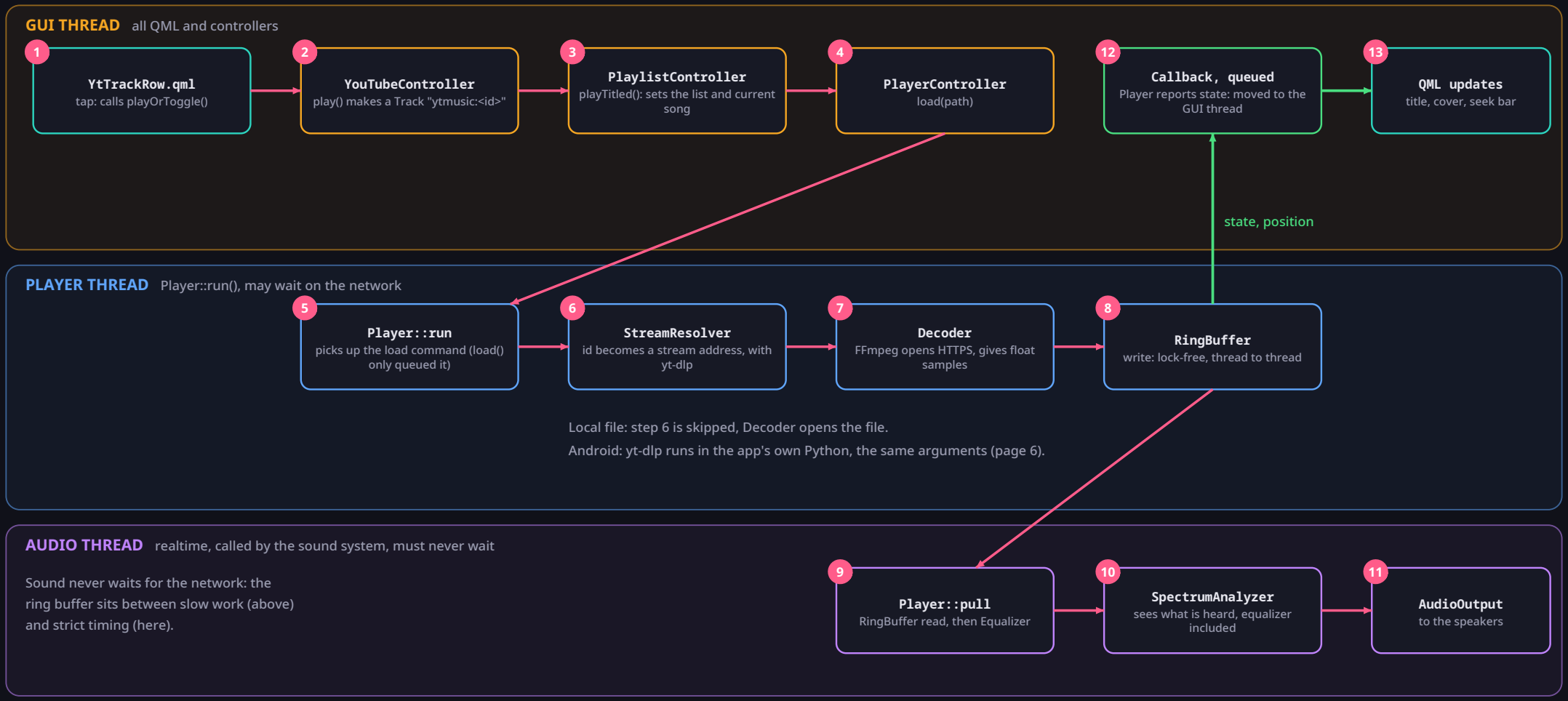
Generated from the code: the names used in each frontend folder, and each wrapper class's #include lines.

Wrapper class	Used by these screens	Backend modules it calls	Other wrapper parts it uses
`PlayerController`	Main, controls, home, layout, library, lyrics, nowplaying, playlist, settingsdialog, youtube	audiooutput, decoder, player, streamresolver	-
`PlaylistController`	controls, layout, lyrics, nowplaying, playlist	playlist, playliststore, streamresolver, track	Files, PlayerController, SettingsController
`PlaylistsController`	Main, dragdrop, dropzone, library, playlist, playlists, youtube	m3u, playlist, playlistlibrary	Files, PlayerController, PlaylistController, SettingsController, YouTubeController
`LibraryController`	detailsdialog, dropzone	metadata, scanner, ytmusicclient	PlaylistController
`LocalLibraryController`	Main, detailsdialog, home, library, settingsdialog	metadata, musiclibrary, tageditor, track	Files, PlaylistController, PlaylistsController, SettingsController, YtMetadataController
`YouTubeController`	Util, dragdrop, home, settingsdialog, youtube	linklibrary, streamresolver, track, ytdlp, ytmusicclient	Files, PlaylistController, SettingsController, YtDlpManager
`YtMetadataController`	detailsdialog	metadata, track, ytmatcher, ytmetadatastore	Files, PlayerController, PlaylistController, SettingsController
`YtDlpManager`	(from C++ only)	httpclient, process, ytdlp	Files
`LyricsController`	lyrics	lyricsprovider, metadata	Files, PlayerController, PlaylistController, SettingsController
`CoverController`	Util	coverfinder, metadata, streamresolver	Files, PlayerController, SettingsController, YtMetadataController
`CoverPalette`	Main	httpclient, metadata	CoverController, PlayerController, PlaylistController
`AlbumArtProvider`	Util	httpclient, metadata	CoverController
`HistoryController`	Main, home, settingsdialog	listeninghistory, streamresolver, track	Files, LocalLibraryController, PlayerController, PlaylistController, PlaylistsController, SettingsController
`SmartPlaylistsController`	home	smartplaylist, streamresolver	Files, HistoryController, LocalLibraryController, PlaylistController
`SessionController`	(from C++ only)	metadata, streamresolver	PlayerController, PlaylistController, PlaylistsController, SettingsController
`SettingsController`	Main, Theme, controls, home, layout, library, lyrics, nowplaying, playlist, settingsdialog, youtube	settings, tsv	Files
`SleepTimerController`	controls	-	PlayerController
`VisualizerController`	nowplaying	spectrum	PlayerController, SettingsController
`NotificationController`	(from C++ only)	httpclient, streamresolver	CoverController, PlayerController, PlaylistController, SettingsController
`SystemMedia`	(from C++ only)	httpclient, track	CoverController, Files, MprisAdaptor, PlayerController, PlaylistController
`MprisAdaptor`	(from C++ only)	metadata	Files, PlayerController, PlaylistController
`Files`	dialogs, settingsdialog	-	-

How to read a row: PlaylistController is used by the playlist screens and others, calls the backend Playlist, and tells PlayerController what to play. Screens can also be Main, Theme and Util. "(from C++ only)" means no QML names it: main.cpp or another class uses it. Each class includes its own list models. AlbumArtProvider is reached by image://albumart/ addresses. YtDlpManager is reached through YouTubeController. Calls inside the backend are not listed here.

Following one song

You tap a YouTube track. Three threads work together, and the screen updates by itself.



When PlayerController changes, others react by themselves

`LyricsController`	fetches the lyrics for the new song	`CoverPalette`	colors the background from the cover
`HistoryController`	records it after enough listening	`SessionController`	remembers the song and position
`SystemMedia`	updates lock screen and media keys	`NotificationController`	shows a song-change notification
`VisualizerController`	reads the spectrum about 60 times a second	`YtMetadataController`	looks up local songs on YouTube

Nothing calls these one by one: each connects to PlayerController's signals itself. So a feature like this can be added without changing the player.

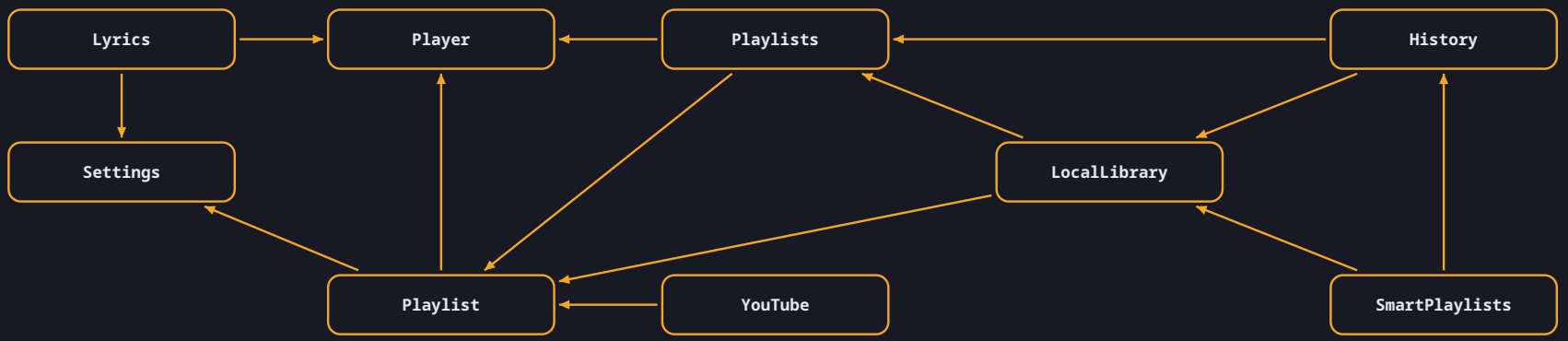
Starting up

main.cpp, in order. Each object needs the ones before it, so the order matters.

1. **Before the app** — Windows only: set the app id, ignore outside Qt settings
2. **moveOldFolders()** — old ChaoticMusicPlayer folders become TunedChaos
3. **QGuiApplication** — Qt starts; interface size applied just before
4. **Android** — save the phone's certificates for HTTPS
5. **SingleInstance** — a second copy wakes the first, then quits
6. **Settings** — SettingsController: everything else reads it
7. **Core** — PlayerController, PlaylistController, LibraryController
8. **Content** — Lyrics, YtMetadata, Covers, YouTube, Playlists, LocalLibrary
9. **Extras** — History, SmartPlaylists, SleepTimer, Notifications, Session, Visualizer
10. **Connect** — settings changes go to the player: fades, EQ, quality...
11. **QML** — image provider added, Main.qml loaded
12. **SystemMedia** — media keys and lock screen
13. **app.exec()** — the app runs

How the controllers depend on each other

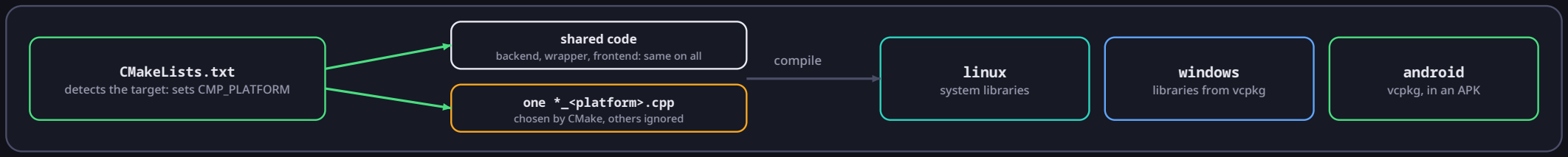
Passed into the constructors in main.cpp. An arrow means "needs".



Settings is passed to most of them; only the main links are drawn.

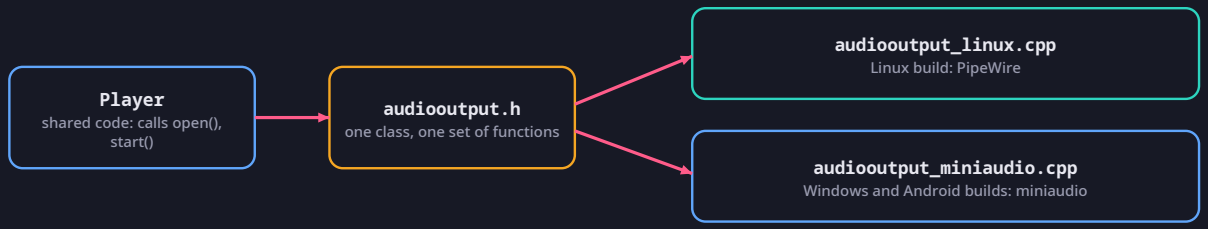
One code base, three platforms

Almost all code is shared. Where a platform differs, one interface hides a small .cpp per platform.



What differs	Shared interface	Linux	Windows	Android
Sound output	`audiooutput.h`	PipeWire	miniaudio (WASAPI)	miniaudio (AAudio)
Running programs	`process.h`	posix_spawn, pipes	CreateProcess, pipes	no programs: Java runner into Python
yt-dlp	`YtDlpManager`	bundled or system; updates itself	yt-dlp.exe; updates itself	Python in the APK (Chaquopy)
Media controls, lock screen	`systemmedia.h`	MPRIS on D-Bus	System Media Transport Controls	MediaPlaybackService, foreground
Song notifications	`notificationcontroller.h`	D-Bus notifications	toast (needs the Start menu id)	none: the media notification
HTTPS certificates	`httpclient, decoder`	the system's	FFmpeg: schannel, Windows' own	phone's certificates, saved to a file
App folders	`Files::configDir() etc.`	~/.config, ~/.local/share	AppData\Local	the app's private storage
Background playing	`Main.qml, service`	window hides, keeps playing	window hides, keeps playing	foreground service
Going full screen	`Main.qml margins`	normal window	normal window	behind the bars, SafeArea margins
Touch	`Theme.touchOnly`	hover buttons, tooltips	hover buttons, tooltips	always-visible dots, tap to play

Example: how one interface works (sound output)



CMake compiles only one of the two .cpp files, so Player needs no #ifdef and never knows which one it got.

All the per-platform files

`audiooutput_*.cpp`	sound output	plus android/ (Java, Python, manifest)
`process_*.cpp`	running programs	and release/ (the packages)
`systemmedia_*.cpp`	media controls	
`notificationcontroller_*.cpp`	notifications	

Same on all three

Paths are UTF-8 with / between folders, even on Windows; QML asks the Files singleton. Layout is picked from the window size, so a phone is just a small window.

To keep it that way

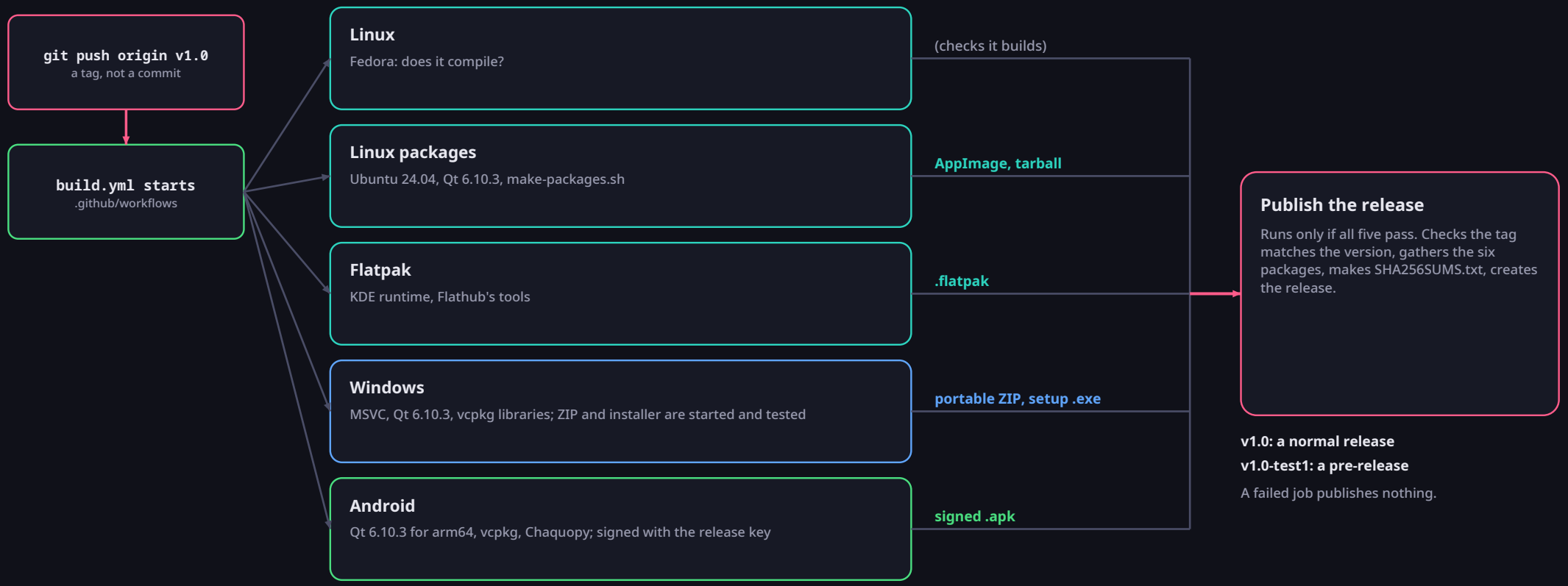
Put features in shared code. Only when the system truly differs: a shared header, one .cpp per platform, listed in CMakeLists.txt.

Small exceptions

A few short #ifdef blocks remain where a whole file would be too much, such as main.cpp, httpclient and decoder.

From code to a release

Ordinary pushes build nothing. Pushing a v* tag starts everything on GitHub.



The six packages, and what each needs

setup .exe
Windows: per-user, no admin rights, Start menu entry

portable ZIP
Windows: extract and run; C++ runtime included

.apk
Android: arm64 phones; Python and yt-dlp inside

AppImage
Linux: one file; chmod +x once

tarball
Linux: install-app.sh, no root

Flatpak
Linux: uses the KDE runtime from Flathub

Where the libraries come from

Linux builds use the system's libraries. Windows and Android build theirs with vcpkg, from the list in release/vcpkg.json, and GitHub caches the result, so only the first run is slow. Qt comes from aqtinstall. The packages bundle what they need, so a user installs nothing else (the Flatpak fetches its KDE runtime once). The Android release key lives in GitHub secrets, never in the repository.